

Reflection: Apple HBR RAG -- Retrieval-Augmented Generation

UTA-PGP-AIABA: Project 1 - Apple HBR RAG Module Project

Created by Brock Frary

2026-09-21

1. Project Overview

This project builds a Retrieval-Augmented Generation (RAG) prototype over an 11-page Harvard Business Review article, "How Apple Is Organized for Innovation." The scenario is a venture capital analyst who needs specific answers out of a dense report without reading the whole thing. Rather than building the RAG pipeline in isolation, the notebook answers the same three questions three different ways -- a bare LLM, a prompt-engineered LLM, and RAG -- so the value of retrieval is visible through direct, run-and-compare evidence rather than asserted. The authorship question ("who wrote this article?") turned out to be the cleanest possible demonstration of that contrast: a model with no access to the document cannot answer it, and only RAG, given the article's own text, answers it correctly with a page citation.

2. Architecture

The pipeline is a straight-through sequence, not a cyclic graph: load, clean, chunk, embed, store, retrieve, generate. The two diagrams below present the same system from two angles -- the end-to-end pipeline itself, and a side-by-side comparison of the three question-answering approaches run against it.

Figure A -- RAG Pipeline Architecture

The source PDF is loaded one Document per page (preserving page-level metadata for later citation), cleaned of repeating footers and typesetting artifacts, split into 512-token chunks with 100-token overlap, embedded, and stored in a Chroma vector index. A similarity retriever pulls the top k=8 chunks for a given question, and a chat completion generates an answer grounded in that context, returned alongside the source page numbers. Both the embedding and chat calls follow the same primary-first, automatic-fallback strategy.

RAG Pipeline -- Source PDF to Grounded, Cited Answer

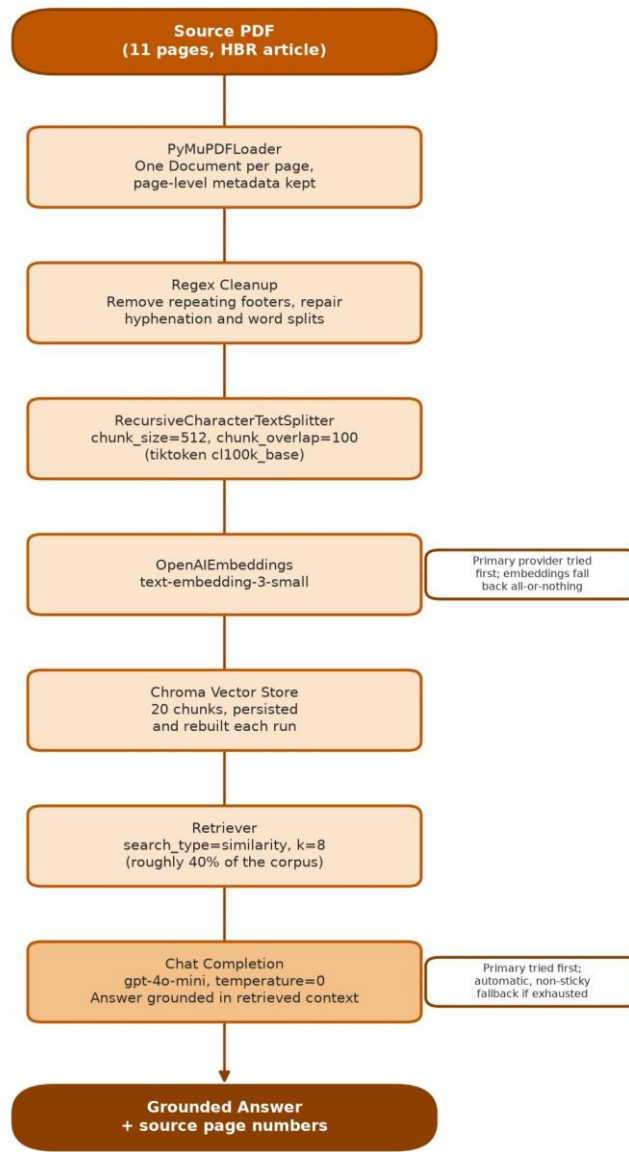
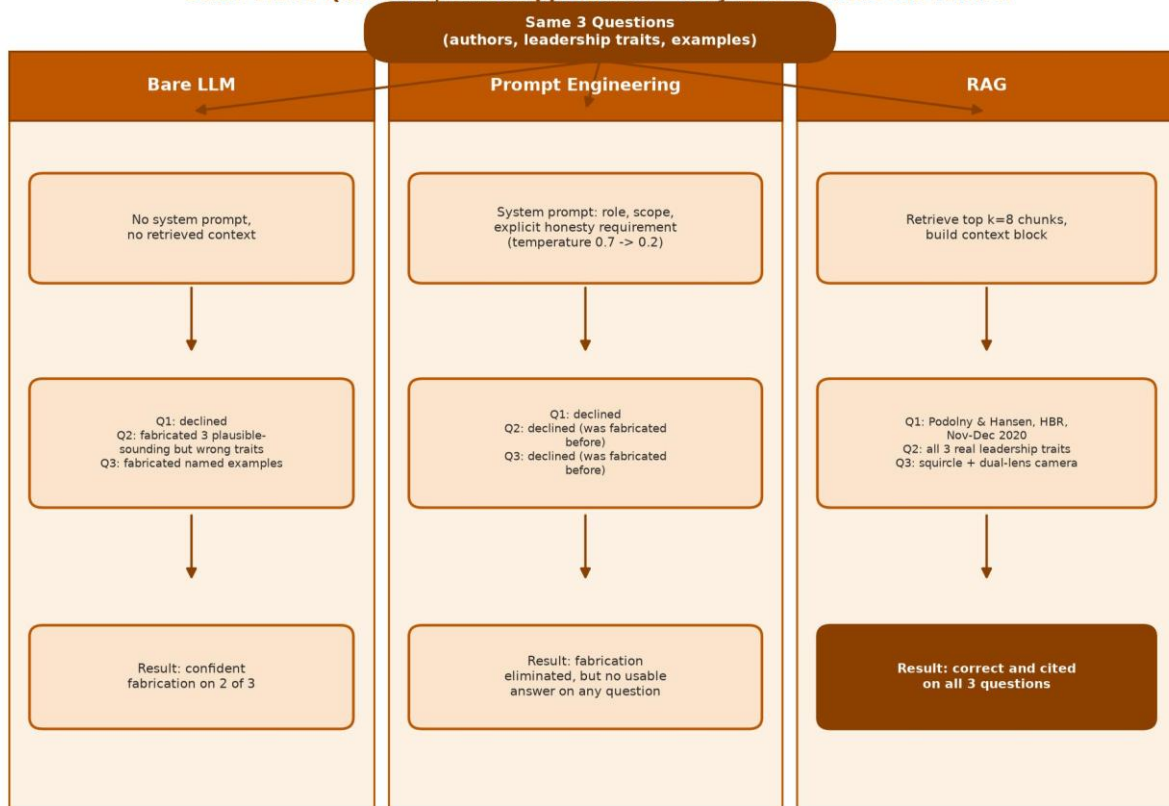


Figure B -- Three-Way Comparison

The same three questions run through all three approaches. The bare LLM either declined or fabricated plausible-sounding but wrong content; adding a system prompt with an explicit honesty requirement eliminated the fabrication but left every question unanswered; only RAG, given the article's own text, answered all three correctly with page citations. The model's raw capability never changed between runs -- what changed was whether it had the right information in front of it.

Same Three Questions, Three Approaches -- Only RAG Grounds the Answer



3. Design Decisions

Provider fallback strategy

Every chat and embedding call tries the primary provider first, with backoff retries on transient failures, and falls back to a secondary provider only once the primary is genuinely exhausted. The fallback is deliberately not sticky - the next call always starts at the primary again -- so a brief primary outage does not permanently shift the notebook onto a different model. Embeddings are the one exception to per-call fallback: since vectors from two different embedding models are not comparable, a single check decides which provider embeds the entire index, so the store is never a mix of the two.

PDF cleanup approach

Inspecting the raw extracted text directly, rather than assuming a generic cleanup recipe would be sufficient, turned up five distinct artifacts: a repeating compliments-notice footer, a repeating reprint-call line, soft hyphens, zero-width spaces inside genuine compound words, and hyphens at line-wraps. The line-wrap case needed the most care: most should be joined and the hyphen dropped, but four genuine compounds (cross-functional, short-term, and two others) happen to fall on a line break and must keep their hyphen -- identified by checking every occurrence against this specific document's actual wording rather than a general rule.

Chunking parameters

chunk_size=512 tokens with chunk_overlap=100 tokens (about 20 percent). The corpus is short -- about 5,300 words after cleanup -- so a smaller chunk size gives finer retrieval granularity than one large chunk per page

would, and the overlap gives enough headroom to avoid splitting a leadership characteristic or example across a chunk boundary.

Retrieval k tuning

k=8, retrieving roughly 40 percent of the entire indexed corpus for every question. This was not the starting value - see Section 4 for the live failure that led here.

RAG prompt design

The RAG prompt explicitly instructs the model to connect concrete details in the retrieved excerpts to the question even when the excerpts do not use the question's exact wording, rather than requiring a literal textual match. A narrative passage describing a specific example rarely repeats the exact words used in an abstract question about that example, so asking the model to bridge that gap in wording, not just supply the right facts, is what actually made the retrieved evidence usable -- see Section 4 for why this mattered as much as the retrieval settings themselves.

4. Challenges and Resolutions

Retrieval failure vs. synthesis failure

The RAG pipeline initially failed the question about specific innovation examples even after raising k. From the outside, a retrieval miss (the right chunk never fetched) and a synthesis failure (the right chunk fetched but the model not using it) look identical -- both come back as "the answer isn't there." Diagnosing this required inspecting the actual retrieved chunks directly, not just re-reading the final answer: the right chunk was present in context, but the model still declined to connect it to the question. The fix was combining k=8 with a revised prompt that explicitly asks the model to bridge wording differences between the question and the source text -- raising k alone was necessary but not sufficient, since the two failure modes needed two different fixes applied together.

Cross-cell CSS does not cascade in a live notebook viewer

An early attempt colored every heading level in the notebook via one shared <style> block. It appeared to work when checked with a full HTML export (nbconvert merges every cell into one continuous document), but a static export is not how the notebook is actually viewed most of the time -- in a live viewer (VS Code, JupyterLab, Colab's own view), each markdown cell renders as its own independent fragment, so the style block only ever reached the one heading sharing its own cell. The fix was scoping color to the title (H1) only, in the same cell as its own style block, and leaving every other heading at the viewer's default color -- and treating this as a reminder that a technique verified only through a merged static export is not actually verified for how the document is normally read.

Oversized title-block images in the exported HTML

The two letterhead logos were sized with a bare HTML height attribute and no width. That rendered correctly in every live notebook viewer, but broke in the actual exported static HTML: the export embeds a standard CSS bundle that resets height to auto for every image, which silently overrides a bare HTML attribute (a low-priority "presentational hint") without needing !important to do it. With no width constraint either, each logo rendered at its full natural pixel size in the file that actually gets submitted. The fix was switching to an inline style attribute on each image, which wins over a selector-based rule in an embedded stylesheet regardless of that rule's specificity -- the same category of lesson as the heading-color issue above: correct in a live viewer is not the same as verified in the actual exported artifact.

A cover image that would not render from a mounted Drive path

The report cover image was originally referenced by its absolute path inside a mounted Google Drive folder. This is not a URL the browser can fetch -- Colab's markdown renderer serves an `<img src>` from a real URL, not from a local or Drive-mounted filesystem path -- so the image showed a broken-image icon on the actual Colab run. This mirrors the two logos, which were external URLs from the start and always rendered correctly; the fix, not yet applied to the final submission by choice, is hosting the cover image externally the same way.

A false-positive secret-pattern match

A routine secret-leak grep for the `sk-` prefix (a common OpenAI key pattern) matched the substring inside the word "risk-taking," which appeared more than once across the bare LLM's hallucinated answers. Documented explicitly as a known non-issue requiring a manual look at any match, rather than trusting the pattern match alone.

5. Trade-offs

The following table summarizes the key trade-offs made during the build:

Decision	Trade-off
k=8 retrieval (roughly 40% of the corpus)	Enough breadth to surface a supporting passage even when it shares little vocabulary with the question; more context tokens sent per call than a narrower k would use
Small chunk size (512 tokens) for a short corpus	Finer retrieval granularity on an 11-page document; would need re-tuning entirely for a much longer or more varied document library
Provider fallback not sticky	A brief primary outage never permanently shifts the notebook onto a different model; every retry sequence re-tries the primary first, even if it just failed
Chroma rebuilt fresh every run	Re-executing the notebook never leaves duplicate or stale chunks behind; no benefit from a persisted index across separate runs
Notebook is Colab-only in its own narrative	Matches the actual grading and deployment environment exactly; the local execution fallback code still works but is deliberately unmentioned in the notebook's own text
Report cover image referenced by Drive path, not re-hosted externally	Matches the project's real Google Drive file location; does not render in Colab's markdown, and was accepted as-is for the final submission rather than spending another Colab round trip on a cosmetic fix

6. Conclusion

The project delivers a working RAG prototype, verified against a live Colab run producing the actual graded submission file. The three-way comparison is the real evidence for the project's central claim, not an assumption: the bare LLM declined or fabricated, prompt engineering traded fabrication for silence, and only retrieval-augmented generation produced correct, page-cited answers to all three questions. Two cosmetic gaps in the final submission -- the report cover image and the orange title color, both added after the Colab run that produced the accepted final files -- were identified and knowingly accepted rather than chased through another round trip, a deliberate scope decision rather than an oversight.

7. Personal Reflections

On testing techniques where they actually run

Two separate bugs this project (the heading-color cascade and the oversized-image sizing) shared the same root lesson: a fix that looks correct in one rendering context is not verified until it is checked in the context that actually matters. A merged HTML export and a live notebook viewer are not the same rendering model, and neither is a live viewer the same as the exact CSS bundle an export tool embeds. The pattern that held up was treating a claimed fix as unverified until checked against the specific artifact it needed to work in, not the most convenient one to test.

On distinguishing failure modes before fixing them

The retrieval-versus-synthesis diagnosis was the single most useful debugging moment in the project. Both failure modes produce the exact same visible symptom -- an answer that says the information isn't there -- and would have been trivially easy to mis-diagnose by only reading final answers and guessing at a fix. Actually inspecting the retrieved chunks directly turned a guess into a specific, correct fix, and confirmed that raising k alone would not have been enough.

On knowing when a submission is actually final

This project kept two genuinely cosmetic issues open at the point the submission was declared final: a cover image that never rendered from its Drive path, and a title color that was added after the Colab export that produced the accepted files. Neither affects correctness, grading, or the actual RAG pipeline's behavior. Naming that trade-off explicitly and choosing to accept it, rather than either silently shipping it unmentioned or chasing a cosmetic fix through another full Colab round trip, felt like the right way to close out a deliverable that was otherwise genuinely done.

8. Build Walkthrough -- Code

This project has no running application to screenshot the way a console-based agent project would -- everything happens inside the notebook itself. In place of screenshots, this section walks through five representative code excerpts from the notebook's own source, in pipeline order, each with a short explanation of what it does and why it is built the way it is.

Figure 1 -- Provider Fallback

Every chat call goes through the primary provider first, with backoff retries on transient failures. The fallback is used only once the primary is genuinely exhausted, and it is not sticky -- the very next call starts at the primary again.

```
RETRYABLE_ERRORS = (RateLimitError, APITimeoutError, APIConnectionError, APIStatusError)
MAX_PRIMARY_ATTEMPTS = 3
BACKOFF_SECONDS = 2

def call_chat_completion(**kwargs):
    """Call the primary provider with backoff retries, then fall back automatically if needed."""
    last_error = None
    for attempt in range(1, MAX_PRIMARY_ATTEMPTS + 1):
        try:
            kwargs.setdefault("model", PRIMARY_MODEL)
            return primary_client.chat.completions.create(**kwargs), "primary"
        except RETRYABLE_ERRORS as e:
            last_error = e
            print(f"[WARN] Primary attempt {attempt}/{MAX_PRIMARY_ATTEMPTS} failed "
                  f"({type(e).__name__}); retrying.")
            time.sleep(BACKOFF_SECONDS * attempt)
    except Exception:
        raise

    if fallback_client is None:
        raise RuntimeError(
            f"Primary provider exhausted after {MAX_PRIMARY_ATTEMPTS} attempts "
            f"({type(last_error).__name__}: {last_error}) and no fallback is configured."
        )

    print("[WARN] Primary exhausted -- switching to fallback provider for this call.")
    kwargs["model"] = FALLBACK_MODEL
    return fallback_client.chat.completions.create(**kwargs), "fallback"
```

Figure 2 -- PDF Cleanup: Repairing Typesetting Artifacts

Inspecting the raw extracted text turned up soft hyphens, zero-width spaces, and line-wrapped words. Most line-wrap hyphens should be joined and dropped, but four genuine compound words (cross-functional, short-term, and two others) happen to fall on a line break and must keep their hyphen -- identified by checking every occurrence against this specific document's actual wording, not assumed from a general rule.

```
GENUINE_HYPHEN_COMPOUNDS = {"cross-functionally", "short-term",
                             "cross-functional", "open-mindedness"}
HYPHEN_LINEBREAK_RE = re.compile(r"(\w+)-\n(\w+)")

def _dehyphenate(match):
    """Join a line-wrapped word, keeping the hyphen only for a known genuine compound."""
    before, after = match.group(1), match.group(2)
    candidate = f"{before}-{after}"
    if candidate.lower() in GENUINE_HYPHEN_COMPOUNDS:
```

```
return candidate
return before + after
```

Figure 3 -- Chunking Configuration

The article is short (about 5,300 words after cleanup), so a smaller chunk size than a typical corpus gives finer retrieval granularity. The 100-token overlap (about 20 percent) gives enough headroom to avoid splitting a leadership characteristic across a chunk boundary.

```
splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base",
    chunk_size=512,
    chunk_overlap=100,
)
```

Figure 4 -- Retriever Configuration

k=8 retrieves roughly 40 percent of the entire indexed corpus for every question. This started at k=4 and was raised after a live acceptance-oracle failure -- see Section 4, Challenges and Resolutions, for the diagnosis that led here.

```
retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 8})
```

Figure 5 -- Grounded RAG Response Function

Retrieves the top-k chunks, builds a context block from them, and explicitly instructs the model to connect concrete details in the context to the question even when the wording differs, rather than requiring a literal textual match. Returns the answer together with the page numbers it drew from, so each answer can be checked directly against the source.

```
def generate_rag_response(user_input, k=8, max_tokens=1024, temperature=0, top_p=0.95):
    """RAG response: retrieves the top-k most similar chunks, grounds the
    answer in them, and returns the source pages."""
    try:
        query_retriever = vectorstore.as_retriever(
            search_type="similarity", search_kwargs={"k": k})
        retrieved_docs = query_retriever.invoke(user_input)

        context = "\n\n".join(
            f"[Page {doc.metadata.get('page', -1) + 1}] {doc.page_content}"
            for doc in retrieved_docs
        )
        rag_prompt = (
            "The context below is excerpted from an article. Answer the question using only "
            "these excerpts, connecting concrete details described in them to the question "
            "even if the excerpts do not use the question's exact wording. If no answer is "
            "present anywhere in the context, say so explicitly rather than guessing.\n\n"
            f"Context:\n{context}\n\nQuestion: {user_input}"
        )

        completion, provider = call_chat_completion(
            messages=[{"role": "user", "content": rag_prompt}],
            max_tokens=max_tokens, temperature=temperature, top_p=top_p,
        )
        answer = sanitize_text(completion.choices[0].message.content)
        source_pages = sorted({doc.metadata.get("page", -1) + 1 for doc in retrieved_docs})
        print(f"[INFO] served by: {provider} | source pages: {source_pages}")
```

```
    return answer, source_pages
except Exception as e:
    return f"[ERROR] RAG response failed: {type(e).__name__}: {e}", []
```